

Taking DCompute to Apple Silicon: Native Metal GPU programming in D

Student

Asadbek Sindarov

Saarland University, Germany

September 3, 2026



Agenda

1. Project Overview
2. Challenges
3. Implementation Design
4. Demo!

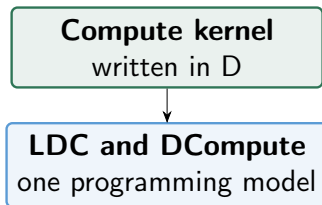


*From D code
to Apple GPU*

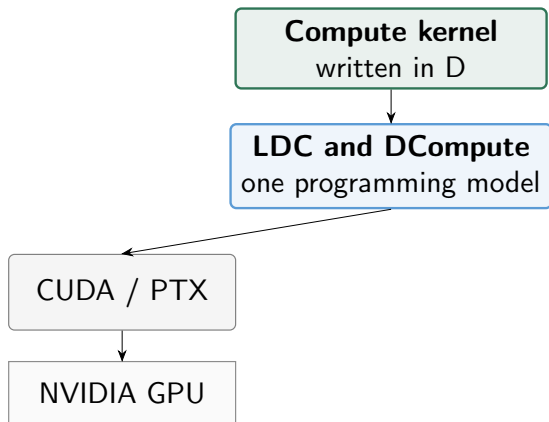
DCompute across GPU platforms

Compute kernel
written in D

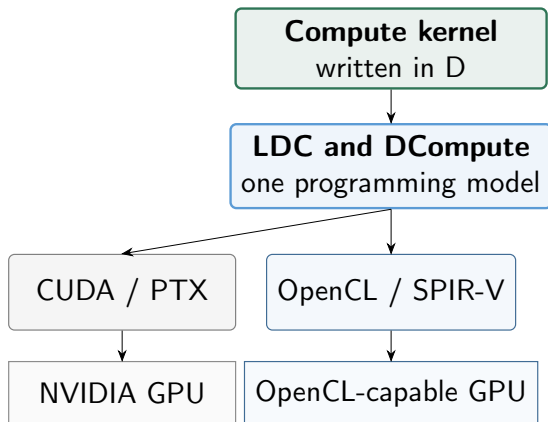
DCompute across GPU platforms



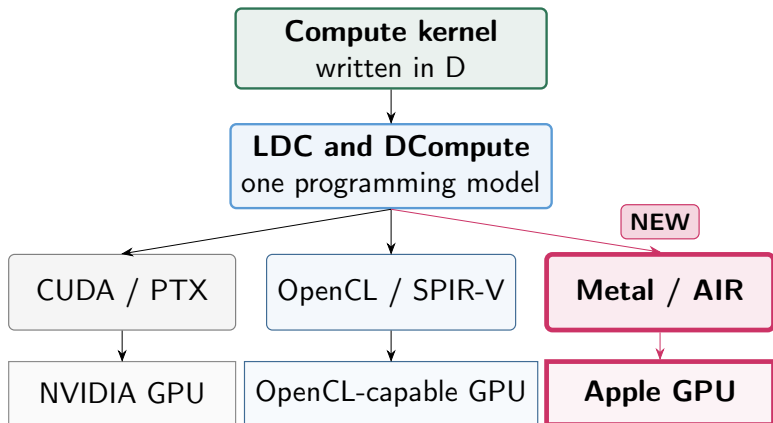
DCompute across GPU platforms



DCompute across GPU platforms



DCompute across GPU platforms



One DCompute programming model, now including Apple GPUs.

The GPU Hello World: Vector Addition

One thread per array element

$$c_i = a_i + b_i$$

- ▶ Input: two arrays, a and b
- ▶ Output: one array, c
- ▶ Thread i computes c_i

*The same kernel runs once
for every array index.*

DCompute kernel

```
@kernel()  
void vectorAdd(  
    GlobalPointer!float a,  
    GlobalPointer!float b,  
    GlobalPointer!float c)  
{  
    auto i = GlobalIndex.x;  
    c[i] = a[i] + b[i];  
}
```

D source code → GPU kernel

D Code and the DCompute Library

Compilation



D code

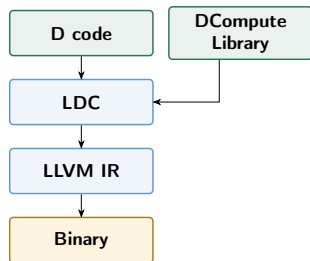
DCompute
library

```
void main() {  
    .....  
    auto program =  
        Program.fromFile(device, "./kernel.air");  
    .....  
    auto queue = Queue(device);  
    auto deviceIn1 =  
        device.makeBuffer!float(a);  
    auto deviceIn2 =  
        device.makeBuffer!float(b);  
    auto deviceOut =  
        device.makeBuffer!float(c);  
    queue.enqueue!(vectorAdd)  
        (globalDims, localDims)  
        (deviceIn1, deviceIn2, deviceOut,  
         width, height);  
    queue.finish();  
}
```

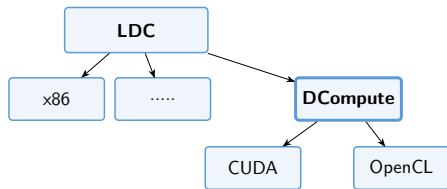
```
@kernel()  
void vectorAdd(  
    GlobalPointer!float a,  
    GlobalPointer!float b,  
    GlobalPointer!float c)  
{  
    auto i = GlobalIndex.x;  
    c[i] = a[i] + b[i];  
}
```

DCompute Targets CUDA and OpenCL

Compilation



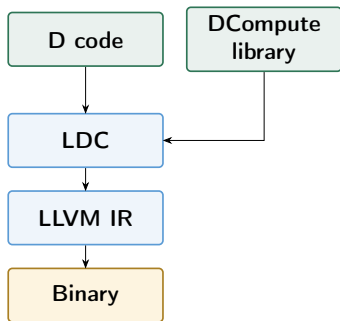
LDC compiles native code for x86 directly, while DCompute targets GPUs through CUDA and OpenCL



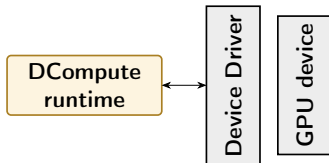
LDC's LLVM backend handles native code generation for various CPUs, while DCompute lowers GPU kernels to CUDA and OpenCL.

The DCompute Runtime

Compilation

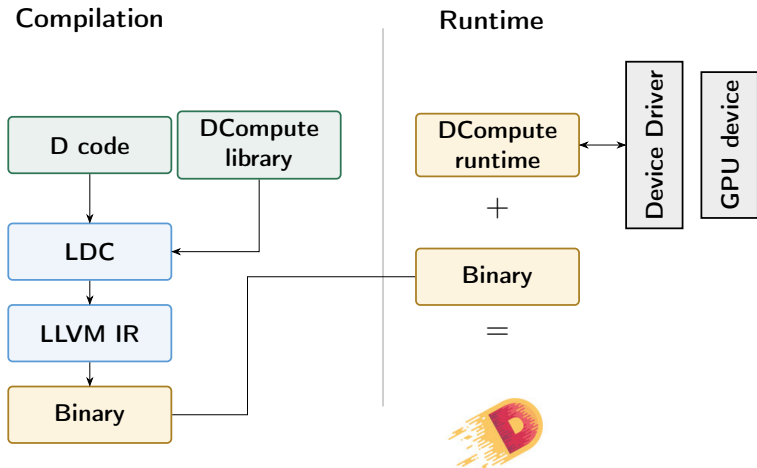


Runtime



- Interacts with the GPU driver
- Handles device context and buffer allocation

Binary + Runtime, Together



Challenges



01

Project scope

The project requires modifications in different parts of the compilation

02

Device Independent Programming

DCompute supports hardware independent programming model.

03

Apple LLVM is closed source

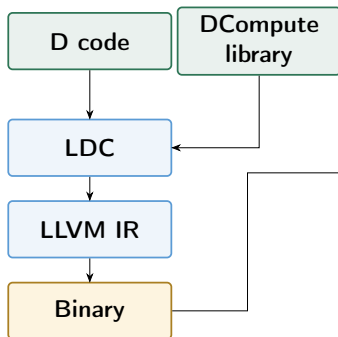
Unlike NVIDIA or AMD, which upstream their LLVM.

04

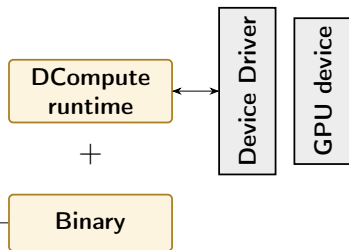
Apple LLVM is older version of upstream LLVM

Apple maintains their LLVM but its version is diverged from the upstream.

Compilation



Runtime



Project involves lots of moving parts across the compilation pipeline.

Challenges

01

Project scope

The project requires modifications in different parts of the compilation



02

Device Independent Programming

DCompute supports hardware independent programming model.

03

Apple LLVM is closed source

Unlike NVIDIA or AMD, which upstream their LLVM.

04

Apple LLVM is older version of upstream LLVM

Apple maintains their LLVM but its version is diverged from the upstream.

How Vector Add is Written in Different Languages

DCompute

```
@kernel()
void vectorAdd(
    GlobalPointer!float a,
    GlobalPointer!float b,
    GlobalPointer!float c)
{
    auto i = GlobalIndex.x;
    c[i] = a[i] + b[i];
}
```

CUDA

```
--global__ void vector_add(
    const float* input_a,
    const float* input_b,
    float* output)
{
    int tid = threadIdx.x
        + blockIdx.x * blockDim.x;

    output[tid] =
        input_a[tid] + input_b[tid];
}
```

OpenCL

```
--kernel void vector_add(
    __global const float* input_a,
    __global const float* input_b,
    __global float* output)
{
    int tid = get_global_id(0);

    output[tid] =
        input_a[tid] + input_b[tid];
}
```

MSL

```
kernel void vector_add(
    device float* input_a [[buffer(0)]],
    device float* input_b [[buffer(1)]],
    device float* output [[buffer(2)]],
    uint tid [[thread_position_in_grid]])
{
    output[tid] =
        input_a[tid] + input_b[tid];
}
```

How Vector Add is Written in Different Languages

DCompute

```
@kernel()
void vectorAdd(
    GlobalPointer!float a,
    GlobalPointer!float b,
    GlobalPointer!float c)
{
    auto i = GlobalIndex.x;
    c[i] = a[i] + b[i];
}
```

Argument
annotations

Metal Shading Language
passes index as an
argument

CUDA

```
--global__ void vector_add(
    const float* input_a,
    const float* input_b,
    float* output)
{
    int tid = threadIdx.x
        + blockIdx.x * blockDim.x;

    output[tid] =
        input_a[tid] + input_b[tid];
}
```

OpenCL

```
--kernel void vector_add(
    __global const float* input_a,
    __global const float* input_b,
    __global float* output)
{
    int tid = get_global_id(0);

    output[tid] =
        input_a[tid] + input_b[tid];
}
```

MSL

```
kernel void vector_add(
    device float* input_a [[buffer(0)]],
    device float* input_b [[buffer(1)]],
    device float* output [[buffer(2)]],
    uint tid [[thread_position_in_grid]])
{
    output[tid] =
        input_a[tid] + input_b[tid];
}
```

Challenges

01

Project scope

The project requires modifications in different parts of the compilation

02

Device Independent Programming

DCompute supports hardware independent programming model.



03

Apple LLVM is closed source

Unlike NVIDIA or AMD, which upstream their LLVM.

04

Apple LLVM is older version of upstream LLVM

Apple maintains their LLVM but its version is diverged from the upstream.

Challenges

01

Project scope

The project requires modifications in different parts of the compilation

02

Device Independent Programming

DCompute supports hardware independent programming model.

03

Apple LLVM is closed source

Unlike NVIDIA or AMD, which upstream their LLVM.

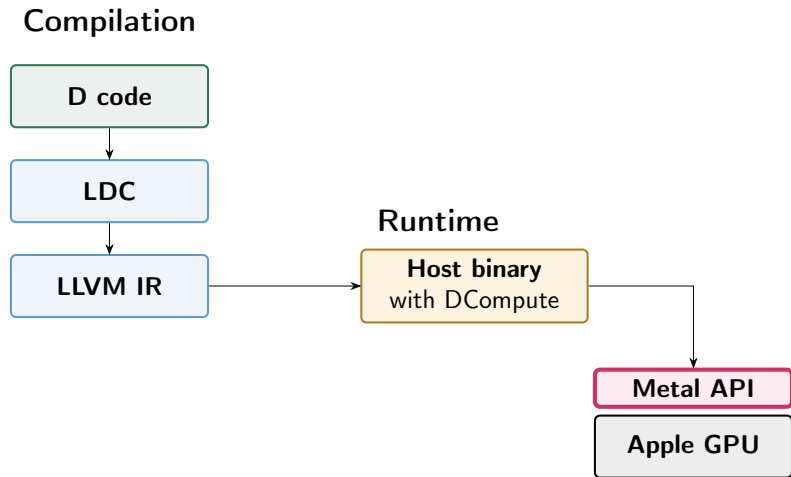
04

Apple LLVM is older version of upstream LLVM

Apple maintains their LLVM but its version is diverged from the upstream.

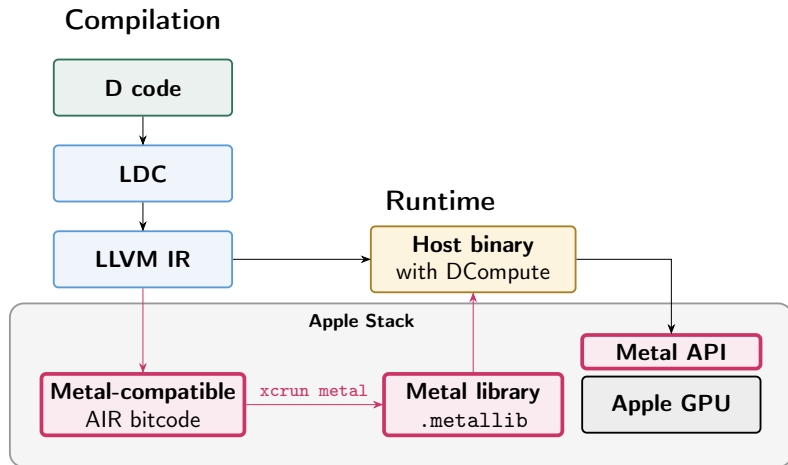


From D Source to Apple GPU



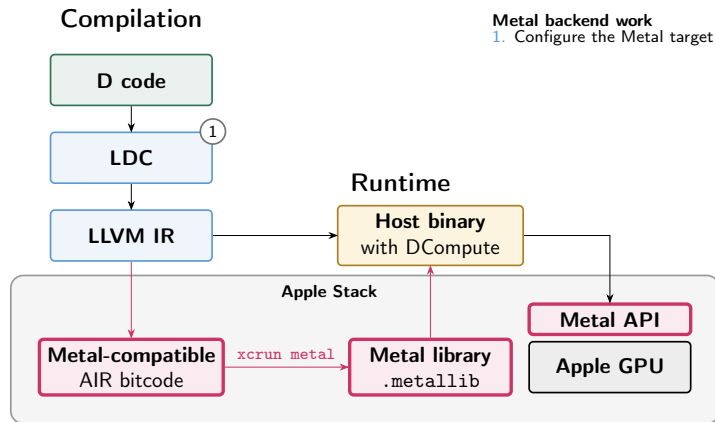
DCompute compiles D kernels into a Metal library and dispatches them through the Metal API.

From D Source to Apple GPU



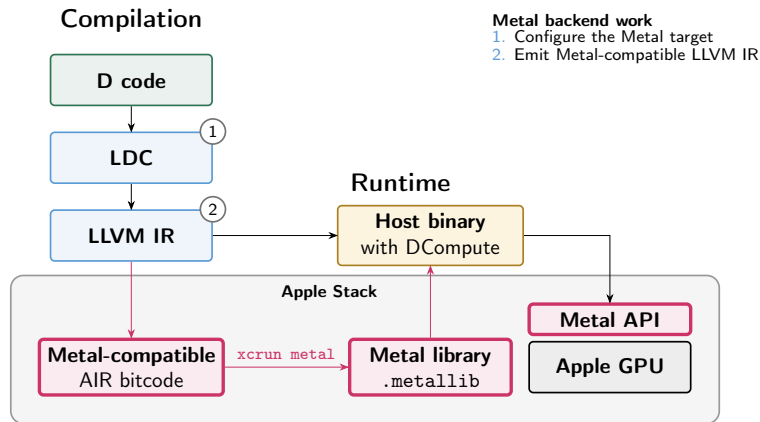
DCompute compiles D kernels into a Metal library and dispatches them through the Metal API.

From D Source to Apple GPU



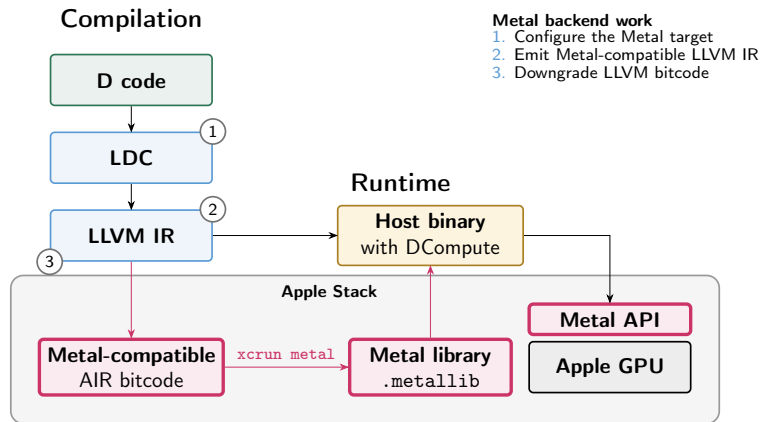
DCompute compiles D kernels into a Metal library and dispatches them through the Metal API.

From D Source to Apple GPU



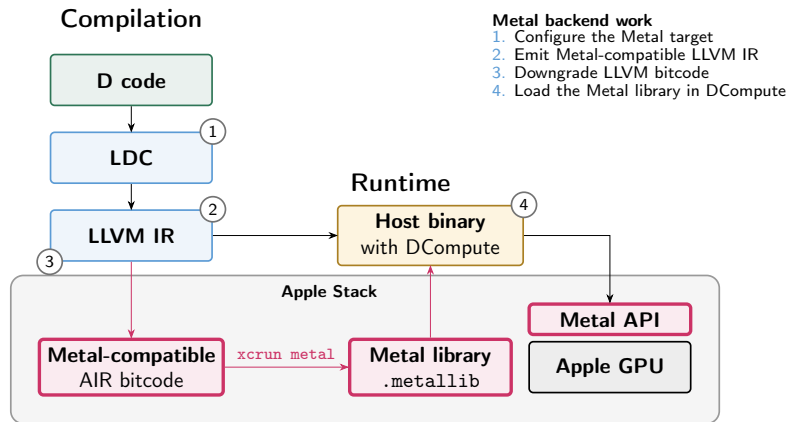
DCompute compiles D kernels into a Metal library and dispatches them through the Metal API.

From D Source to Apple GPU



DCompute compiles D kernels into a Metal library and dispatches them through the Metal API.

From D Source to Apple GPU



DCompute compiles D kernels into a Metal library and dispatches them through the Metal API.

Configuring the Metal Target

Defining the Architecture

- ▶ Every backend requires a `DComputeTarget` subclass to define its properties.
- ▶ **Target Triple:** We explicitly enforce `air64_v28-apple-macosx` to ensure LLVM emits valid Apple IR.
- ▶ **Address Spaces:** Metal strictly separates memory. We map DCompute's generic memory model to Metal's exact hardware address spaces:
 - ▶ Private = 0
 - ▶ Device/Global = 1
 - ▶ Constant = 2
 - ▶ Threadgroup = 3

```
class TargetMetal : public DComputeTarget {
public:
    TargetMetal(llvm::LLVMContext &c, int version)
        : DComputeTarget(c, version, ID::Metal,
            "metal", "air", createMetalABI(),
            // Address Space Map:
            // [Private, Global, Shared, Constant, Generic]
            {{0, 1, 3, 2, 0}})
    {
        std::string triple =
            "air64_v28-apple-macosx26.0.0";

        _ir->module.setTargetTriple(triple);

        auto floatABI = ::FloatABI::Hard;
        targetMachine = createTargetMachine(
            triple, "arm64", "", {},
            ExplicitBitness::M64, floatABI,
            llvm::Reloc::Static,
            llvm::CodeModel::Small,
            codeGenOptLevel(), false);

        // ... DataLayout initialization ...
    }
};
```

Binding Apple IR Intrinsics

Masking Intrinsics

- ▶ Unlike CUDA and AMD, we must manually expose Apple's internal AIR intrinsics to the D front end.
- ▶ We use `pragma(LDC_intrinsic, ...)` to map D function calls directly to LLVM IR nodes.
- ▶ This provides a zero-cost abstraction for querying grid and threadgroup dimensions at runtime.
- ▶ These primitives are seamlessly wrapped by the target-agnostic `GlobalIndex` struct.

1. Exposing the IR Nodes

```
pragma(LDC_intrinsic, "air.get_global_id.i32")
uint thread_position_in_grid(uint dim);

pragma(LDC_intrinsic, "air.get_local_id.i32")
uint thread_position_in_threadgroup(uint dim);

// ... other dimension queries ...
```

2. The Target-Agnostic Wrapper

```
struct GlobalIndex
{
    pragma(inline,true);
    @property static size_t x()()
    {
        if(__dcompute_reflect(ReflectTarget.OpenCL, 0))
            return ocl.get_global_id(0);
        else if(__dcompute_reflect(ReflectTarget.CUDA, 0))
            return cuda.ctaid_x() * cuda.ntid_x()
                + cuda.tid_x();
        else if(__dcompute_reflect(ReflectTarget.Metal, 0))
            return metal.thread_position_in_grid(0);
        else
            assert(0);
    }

    // ... y(), z(), and linear() implementations ...
}
```

Observations in Metal LLVM IR

Key Metadata Nodes

- **Target Triple:**
Dictates the specific architecture and OS.

```
; ModuleID = 'test.metal'
source_filename = "test.metal"
target datalayout = "e-p:64:64:64-i1:8:8-..."
target triple = "air64_v28-apple-macosx26.0.0"

; Function Attrs: argmemonly mustprogress norecurse
define void @vectorAdd(
  ptr addrspace(1) nocapture writeonly %0,
  ptr addrspace(1) nocapture readonly %1,
  ptr addrspace(1) nocapture readonly %2) #0
{
  %4 = tail call @i32 @air.get_global_id.i32(i32 0)
  %5 = zext i32 %4 to i64
  ; ... GEP and fadd execution graph ...
  ret void
}

; --- Apple Specific Metadata ---
!llvm.module.flags = !{!0, !1, !2, !3, !4}
!air.kernel = !{!9}
!air.compile_options = !{!15, !16, !17}
!air.version = !{!19}

!9 = !{ptr @vectorAdd, !10, !11}
!11 = !{!12, !13, !14}
!12 = !{i32 0, !"air.buffer", !"air.location_index", i32 0, ...}
!13 = !{i32 1, !"air.buffer", !"air.location_index", i32 1, ...}
```

Observations in Metal LLVM IR

Key Metadata Nodes

- ▶ **Target Triple:**
Dictates the specific architecture and OS.
- ▶ **Kernel Metadata:**
Identifies the compute entry point for Metal.

```
; ModuleID = 'test.metal'
source_filename = "test.metal"
target datalayout = "e-p:64:64:64-i1:8:8-..."
target triple = "air64_v28-apple-macosx26.0.0"

; Function Attrs: argmemonly mustprogress norecurse
define void @vectorAdd(
  ptr addrspace(1) nocapture writeonly %0,
  ptr addrspace(1) nocapture readonly %1,
  ptr addrspace(1) nocapture readonly %2) #0
{
  %4 = tail call @i32_get_global_id.i32(i32 0)
  %5 = zext i32 %4 to i64
  ; ... GEP and fadd execution graph ...
  ret void
}

; --- Apple Specific Metadata ---
!llvm.module.flags = !{!0, !1, !2, !3, !4}
!air.kernel = !{!9}
!air.compile_options = !{!15, !16, !17}
!air.version = !{!19}

!9 = !{ptr @vectorAdd, !10, !11}
!11 = !{!12, !13, !14}
!12 = !{i32 0, !"air.buffer", !"air.location_index", i32 0, ...}
!13 = !{i32 1, !"air.buffer", !"air.location_index", i32 1, ...}
```

Observations in Metal LLVM IR

Key Metadata Nodes

- ▶ **Target Triple:**
Dictates the specific architecture and OS.
- ▶ **Kernel Metadata:**
Identifies the compute entry point for Metal.
- ▶ **Buffer Annotations:**
Maps LLVM pointers to physical GPU buffers.

```
; ModuleID = 'test.metal'
source_filename = "test.metal"
target datalayout = "e-p:64:64:64-i1:8:8-..."
target triple = "air64_v28-apple-macosx26.0.0"

; Function Attrs: argmemonly mustprogress noreturn
define void @vectorAdd(
  ptr addrspace(1) nocapture writeonly %0,
  ptr addrspace(1) nocapture readonly %1,
  ptr addrspace(1) nocapture readonly %2) #0
{
  %4 = tail call @air.get_global_id.i32(i32 0)
  %5 = zext i32 %4 to i64
  ; ... GEP and fadd execution graph ...
  ret void
}

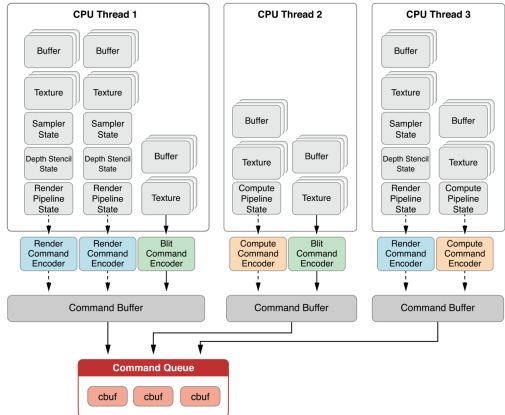
; --- Apple Specific Metadata ---
!llvm.module.flags = !{!0, !1, !2, !3, !4}
!air.kernel = !{!9}
!air.compile_options = !{!15, !16, !17}
!air.version = !{!19}

!9 = !{ptr @vectorAdd, !10, !11}
!11 = !{!12, !13, !14}
!12 = !{i32 0, !"air.buffer", !"air.location_index", i32 0, ...}
!13 = !{i32 1, !"air.buffer", !"air.location_index", i32 1, ...}
```

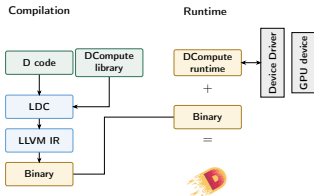
Metal-API

We use Objective-C Metal bindings and it follows the following execution model:

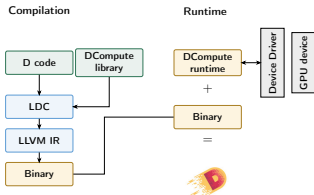
- ▶ Initialize GPU device and load compute kernel.
- ▶ Create pipeline state and command queue.
- ▶ Set up data buffers and command buffer.
- ▶ Encode commands, commit for execution, and optionally wait for completion.



Summary



Summary



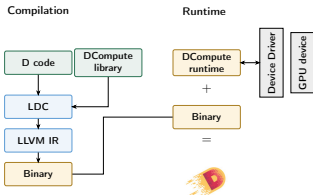
DCompute

```
@kernel()
void vectorAdd(
    GlobalPointer!float a,
    GlobalPointer!float b,
    GlobalPointer!float c)
{
    auto i = GlobalIndex.x;
    c[i] = a[i] + b[i];
}
```

MSL

```
kernel void vector_add(
    device float* a [[buffer(0)]],
    device float* b [[buffer(1)]],
    device float* c [[buffer(2)]]
    uint tid
    [[thread_position_in_grid]])
{
    c[tid] = a[tid]+b[tid];
}
```

Summary

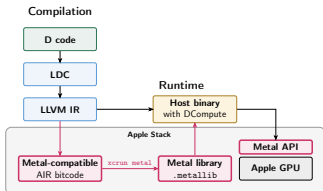


DCompute

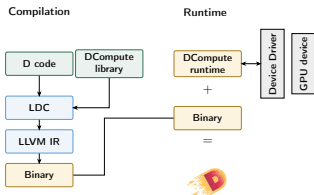
```
@kernel()  
void vectorAdd(  
    GlobalPointer!float a,  
    GlobalPointer!float b,  
    GlobalPointer!float c)  
{  
    auto i = GlobalIndex.x;  
    c[i] = a[i] + b[i];  
}
```

MSL

```
kernel void vector_add(  
    device float* a [[buffer(0)]]  
    device float* b [[buffer(1)]]  
    device float* c [[buffer(2)]]  
    uint tid  
    [[thread_position_in_grid]])  
{  
    c[tid] = a[tid]+b[tid];  
}
```



Summary



DCompute

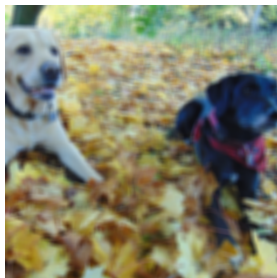
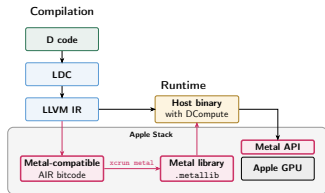
```

@kernel()
void vectorAdd(
    GlobalPointer!float a,
    GlobalPointer!float b,
    GlobalPointer!float c)
{
    auto i = GlobalIndex.x;
    c[i] = a[i] + b[i];
}
  
```

MSL

```

kernel void vector_add(
    device float* a [[buffer(0)]],
    device float* b [[buffer(1)]],
    device float* c [[buffer(2)]],
    uint tid
    [[thread_position_in_grid]])
{
    c[tid] = a[tid]+b[tid];
}
  
```



Thank You!

Any Questions?

Contact:

aosindarov@gmail.com

Source Code:

LDC: <https://github.com/asindarov/ldc>

DCompute: <https://github.com/asindarov/dcompute>

LLVM Downgrader:

https://github.com/asindarov/llvm_downgrader